RAPTOR DEFENSE V2.0 // HANDBOOK

DICTIONNAIRE OFFICIEL DES STRUCTURES ET FONCTIONS // RAPTOR V2.0

RÉFÉRENTIEL TECHNIQUE D'INGÉNIERIE ET D'API (CONSOLIDÉ)

==========
SECTION 1 : MODÈLE DE DONNÉES UNIVERSEL (src/core/raptor_data.c)
==========

Fonction 01 : raptor_data.c – raptor_data_context_create – Allocation du contexte • Signature C : RaptorReportContext* raptor_data_context_create(void); • Utilité : Alloue dynamiquement en mémoire vive et initialise la structure centrale du rapport destinée à recevoir les données des greffons.

Fonction 02 : raptor_data.c – raptor_data_section_add – Création de section • Signature C : RaptorSection* raptor_data_section_add(RaptorReportContext* ctx, const char* id, const char* title); • Utilité : Ajoute un conteneur thématique autonome dans le contexte pour regrouper les cartes, tables et alertes produites par un greffon.

Fonction 03 : raptor_data.c – raptor_data_card_add – Insertion de métrique clé • Signature C : bool raptor_data_card_add(RaptorSection* sec, const char* label, const char* val, float pct, RaptorSeverity sev); • Utilité : Enregistre un indicateur synthétique (valeur, pourcentage, niveau de criticité) affiché sous forme de tuile métrique dans l'interface.

Fonction 04 : raptor_data.c – raptor_data_table_create – Initialisation de tableau • Signature C : RaptorTable* raptor_data_table_create(RaptorSection* sec, const char* title, size_t col_count, const char** col_names); • Utilité : Instancie une structure de tableau bidimensionnel avec ses en-têtes de colonnes au sein d'une section donnée.

Fonction 05 : raptor_data.c – raptor_data_table_row_add – Insertion de ligne de données • Signature C : bool raptor_data_table_row_add(RaptorTable* table, const char** cell_texts, const RaptorSeverity* cell_sevs); • Utilité : Ajoute une ligne complète de cellules textuelles associées à des codes de couleur/sévérité dans le tableau spécifié.

Fonction 06 : raptor_data.c – raptor_data_alert_add – Notification de sécurité • Signature C : bool raptor_data_alert_add(RaptorSection* sec, RaptorSeverity sev, const char* message); • Utilité : Consigne un message d'avertissement ou de confirmation dans la section avec un niveau d'alerte défini (OK, AVERTISSEMENT, CRITIQUE).

Fonction 07 : raptor_data.c – raptor_data_get_section_count – Dénombrement des sections • Signature C : size_t raptor_data_get_section_count(const RaptorReportContext* ctx); • Utilité : Renvoie le nombre total de sections actuellement peuplées dans le contexte de rapport actif.

Fonction 08 : raptor_data.c – raptor_data_get_section – Accès sécurisé par index • Signature C : const RaptorSection* raptor_data_get_section(const RaptorReportContext* ctx, size_t index); • Utilité : Fournit un pointeur en lecture seule vers une section précise à partir de son rang séquentiel dans la collection.

Fonction 09 : raptor_data.c – raptor_data_context_clear – Purge dynamique du contexte • Signature C : void raptor_data_context_clear(RaptorReportContext* ctx); • Utilité : Libère récursivement l'ensemble des sections, tables et alertes tout en maintenant le conteneur principal alloué pour le Live Monitoring 3s.

Fonction 10 : raptor_data.c – raptor_data_context_destroy – Destruction totale • Signature C : void raptor_data_context_destroy(RaptorReportContext* ctx); • Utilité : Désalloue complètement l'arbre de mémoire du contexte et libère le pointeur racine lors de l'arrêt définitif de l'application.

==========
SECTION 2 : REGISTRE DE GREFFONS ET BUS SYSTÈME (src/core/raptor_registry.c)
==========

Fonction 11 : raptor_registry.c – raptor_registry_init – Initialisation du bus • Signature C : void raptor_registry_init(void); • Utilité : Prépare la table interne d'enregistrement et réinitialise les compteurs de modules d'inspection disponibles.

Fonction 12 : raptor_registry.c – raptor_registry_register – Enregistrement de greffon • Signature C : bool raptor_registry_register(const RaptorPlugin* plugin); • Utilité : Inscrit un greffon d'audit dans la table globale sans nécessiter de modification des fichiers sources du noyau central.

Fonction 13 : raptor_registry.c – raptor_registry_run_all – Déclenchement séquentiel • Signature C : bool raptor_registry_run_all(RaptorReportContext* ctx); • Utilité : Parcourt l'ensemble des greffons actifs et exécute leur méthode

`run_audit` pour peupler le contexte en une seule passe.

Fonction 14 : raptor_registry.c – raptor_registry_get_count – Comptage des greffons • Signature C : size_t raptor_registry_get_count(void); • Utilité : Retourne le nombre total de greffons actuellement enregistrés et disponibles pour les audits.

Fonction 15 : raptor_registry.c – raptor_registry_get_by_idx – Accès greffon par index • Signature C : const RaptorPlugin* raptor_registry_get_by_idx(size_t index); • Utilité : Permet d'interroger la configuration, le nom et les métadonnées d'un greffon via son index dans le registre.

Fonction 16 : raptor_registry.c – raptor_registry_cleanup – Déchargement global • Signature C : void raptor_registry_cleanup(void); • Utilité : Déclenche l'appel de nettoyage `cleanup()` sur chaque greffon et vide la table interne des pointeurs de modules.

==========
SECTION 3 : MOTEUR D'ANALYSE DIFFÉRENTIELLE (src/core/raptor_differential.c)
==========

Fonction 17 : raptor_differential.c – raptor_diff_init – Initialisation différentielle • Signature C : void raptor_diff_init(void); • Utilité : Configure les structures mémoire destinées à conserver l'empreinte historique nécessaire au calcul des dérives temporelles.

Fonction 18 : raptor_differential.c – raptor_diff_compute – Calcul des dérivées RAM • Signature C : bool raptor_diff_compute(const RaptorReportContext* current, RaptorSection* out_diff_sec); • Utilité : Compare l'état T(n) avec l'état T(n-1), identifie les fuites de mémoire (Mo/min) et peuple la section du cahier différentiel.

Fonction 19 : raptor_differential.c – raptor_diff_snapshot_save – Sauvegarde d'instantané • Signature C : void raptor_diff_snapshot_save(const RaptorReportContext* current); • Utilité : Copie et fige les métriques de processus actuelles en tant que nouvel état de référence pour les comparaisons futures.

Fonction 20 : raptor_differential.c – raptor_diff_cleanup – Nettoyage différentiel • Signature C : void raptor_diff_cleanup(void); • Utilité : Libère l'intégralité des instantanés historiques archivés en RAM.

==========
SECTION 4 : JOURNALISATION FORENSIQUE PERMANENTE (src/report/report_audit_log.c)
==========

Fonction 21 : report_audit_log.c – audit_log_get_system_btime – Extraction du boot time • Signature C : static unsigned long long audit_log_get_system_btime(void); • Utilité : Lit `/proc/stat` et extrait le timestamp Unix `btime` correspondant à l'instant de démarrage matériel de la machine hôte.

Fonction 22 : report_audit_log.c – audit_log_compute_start_time – Calcul de spawn • Signature C : static void audit_log_compute_start_time(int pid, char* out_buf, size_t buf_sz); • Utilité : Parse le 22e champ (`starttime`) de `/proc/[PID]/stat` et convertit les ticks processeur en heure locale absolue de démarrage (HH:MM:SS).

Fonction 23 : report_audit_log.c – audit_log_get_next_sequence_number – Séquençage auto • Signature C : static int audit_log_get_next_sequence_number(void); • Utilité : Parcourt `AUDIT.TXT` à la recherche du motif `AUDIT NO #XX` pour déterminer et retourner le prochain numéro incrémental unique.

Fonction 24 : report_audit_log.c – raptor_audit_log_capture_target – Capture pré-mortem • Signature C : bool raptor_audit_log_capture_target(int pid, RaptorIncidentDetail* out_detail); • Utilité : Extrait dynamiquement toutes les preuves du processus dans `/proc` (binaire, CWD, start time, PPID, commande parent, CWD parent) avant arrêt.

Fonction 25 : report_audit_log.c – raptor_audit_log_commit – Écriture persistante • Signature C : bool raptor_audit_log_commit(const RaptorIncidentDetail* detail, const char* action_status); • Utilité : Formate et écrit le bloc d'audit horodaté dans `AUDIT.TXT` en mode ajout permanent (`a+`) avec forçage d'écriture physique sur disque.

Fonction 26 : report_audit_log.c – raptor_audit_log_notify_detection – Alerte temps réel • Signature C : bool raptor_audit_log_notify_detection(int pid, const char* classification); • Utilité : Déclenche l'écriture immédiate d'un audit dès qu'une anomalie heuristique apparaît, avec filtrage dédoublonné pour éviter la saturation du fichier.

Fonction 27 : report_audit_log.c – raptor_audit_log_forget_pid – Purge du cache anti-flood • Signature C : void raptor_audit_log_forget_pid(int pid); • Utilité : Supprime un identifiant PID de la table des processus déjà logués lors de sa destruction, autorisant la traque immédiate en cas de réapparition.

==========
SECTION 5 : GREFFON PROCESSUS, MÉMOIRE ET RWX (src/plugins/plugin_memory_rwx.c)
==========

Fonction 28 : plugin_memory_rwx.c – proc_mem_is_trusted_path – Filtrage des chemins sûrs • Signature C : static bool proc_mem_is_trusted_path(const char* exe_path); • Utilité : Valide si le binaire provient strictement des répertoires systèmes légitimes (`/usr/bin/` , `/usr/lib/` , `/bin/` , `/sbin/` , `/app/`).

Fonction 29 : plugin_memory_rwx.c – proc_mem_is_legit_jit – Détection des moteurs JIT • Signature C : static bool proc_mem_is_legit_jit(const char* comm, const char* exe); • Utilité : Écarte les faux positifs en reconnaissant les interpréteurs légitimes allouant de la mémoire exécutable (navigateurs, Python, machines virtuelles).

Fonction 30 : plugin_memory_rwx.c – proc_mem_scan_maps_rwx – Inspection des pages mémoire • Signature C : static bool proc_mem_scan_maps_rwx(uint32_t pid, bool is_jit, bool is_trusted); • Utilité : Analyse `/proc/[PID]/maps` pour repérer les segments illégitimes possédant simultanément les droits Read-Write-Execute (`rwx`).

Fonction 31 : plugin_memory_rwx.c – proc_mem_read_stats – Empreinte RAM du processus • Signature C : static bool proc_mem_read_stats(uint32_t pid, char* out_comm, size_t comm_len, char* out_exe, size_t exe_len, uint64_t* out_rss); • Utilité : Lit le nom d'exécutable, résout le lien `/proc/[PID]/exe` et calcule la consommation résidente RSS en mémoire physique.

Fonction 32 : plugin_memory_rwx.c – proc_mem_run_audit – Exécution de l'audit mémoire • Signature C : static bool proc_mem_run_audit(RaptorReportContext* ctx); • Utilité : Parcourt l'arborescence `/proc` , audite la mémoire de chaque tâche active et consigne les résultats dans la section dédiée du rapport.

Fonction 33 : plugin_memory_rwx.c – proc_mem_register – Enregistrement automatique • Signature C : **attribute**((constructor)) static void proc_mem_register(void); • Utilité : Enregistre le greffon mémoire auprès du registre central dès le chargement du binaire en mémoire par le système d'exploitation.

==========
SECTION 6 : GREFFON SURVEILLANCE RÉSEAU (src/plugins/plugin_network_sockets.c)
==========

Fonction 34 : plugin_network_sockets.c – net_sock_build_inode_cache – Résolution Inode/PID • Signature C : static bool net_sock_build_inode_cache(void); • Utilité : Analyse tous les descripteurs ouverts dans `/proc/[PID]/fd` pour construire la table de correspondance rapide entre numéros d'inodes et PID.

Fonction 35 : plugin_network_sockets.c – net_sock_resolve_pid – Recherche propriétaire O(1) • Signature C : static uint32_t net_sock_resolve_pid(uint64_t target_inode); • Utilité : Retourne instantanément l'identifiant PID associé à un socket réseau à partir de son numéro d'inode noyau.

Fonction 36 : plugin_network_sockets.c – net_sock_decode_hex_endpoint – Décodage IP/Port • Signature C : static void net_sock_decode_hex_endpoint(const char* hex_ip, uint32_t hex_port, char* out_ip, size_t ip_len, uint16_t* out_port); • Utilité : Convertit la représentation hexadécimale brute (format endianess noyau) en adresse IP décimale lisible et port réseau standard.

Fonction 37 : plugin_network_sockets.c – net_sock_parse_file – Analyse des tables réseau • Signature C : static void net_sock_parse_file(const char* net_path, const char* proto, RaptorTable* out_table); • Utilité : Parse `/proc/net/tcp` et `/proc/net/udp` pour répertorier les ports en écoute (`LISTEN`) et les canaux établis (`ESTABLISHED`).

Fonction 38 : plugin_network_sockets.c – net_sock_run_audit – Exécution de l'audit réseau • Signature C : static bool net_sock_run_audit(RaptorReportContext* ctx); • Utilité : Orchestre la collecte des flux réseau, résout les processus propriétaires et libère le cache des descripteurs après analyse.

Fonction 39 : plugin_network_sockets.c – net_sock_register – Enregistrement automatique • Signature C : **attribute**((constructor)) static void net_sock_register(void); • Utilité : Inscrit le greffon réseau dans le registre central dès l'initialisation du binaire exécutable.

==========
SECTION 7 : GREFFON HEURISTIQUE & LIGNÉE PARENT (src/plugins/plugin_heuristics_expert.c)
==========

Fonction 40 : plugin_heuristics_expert.c – heuristics_is_suspicious_path – Détection chemins volatils • Signature C : static bool heuristics_is_suspicious_path(const char* path); • Utilité : Identifie si un processus s'exécute depuis des zones temporaires ou volatiles suspectes (`/tmp` , `/var/tmp` , `/dev/shm` , `/run/user`).

Fonction 41 : plugin_heuristics_expert.c – heuristics_is_system_masquerade – Détection d'usurpation • Signature C : static bool heuristics_is_system_masquerade(const char* comm, const char* exe_target); • Utilité : Repère les processus non officiels usurpant le nom de démons critiques (`systemd` , `sshd` , `cron` , `kworker`) hors des dossiers systèmes.

Fonction 42 : plugin_heuristics_expert.c – heuristics_get_parent_info – Traque généalogique (PPID) • Signature C : static void heuristics_get_parent_info(const char* pid_str, char* out_ppid, size_t ppid_sz, char* out_pcomm, size_t pcomm_sz, char* out_pcmd, size_t pcmd_sz); • Utilité : Extrait le PID parent via `/proc/[PID]/stat` et reconstitue la ligne de commande exacte et l'exécutable créateur depuis `/proc/[PPID]/cmdline` .

Fonction 43 : plugin_heuristics_expert.c – heuristics_check_process – Évaluation unitaire • Signature C : static void heuristics_check_process(const char* pid_str, size_t* out_masquerades, size_t* out_deleted, size_t* out_suspicious_paths, RaptorTable* tbl); • Utilité : Exécute l'ensemble des filtres heuristiques sur un processus, remplit la table d'incident et déclenche la notification persistante dans `AUDIT.TXT` .

Fonction 44 : plugin_heuristics_expert.c – plugin_heuristics_init – Initialisation greffon • Signature C : static bool plugin_heuristics_init(void); • Utilité : Prépare les états internes et structures du greffon heuristique.

Fonction 45 : plugin_heuristics_expert.c – plugin_heuristics_run_audit – Exécution globale heuristique • Signature C : static bool plugin_heuristics_run_audit(RaptorReportContext* ctx); • Utilité : Parcourt `/proc`, identifie les menaces comportementales, compile les cartes d'usurpation et qualifie la criticité défensive globale.

Fonction 46 : plugin_heuristics_expert.c – plugin_heuristics_cleanup – Nettoyage greffon • Signature C : static void plugin_heuristics_cleanup(void); • Utilité : Libère les structures locales allouées lors de l'exécution de l'audit.

Fonction 47 : plugin_heuristics_expert.c – plugin_heuristics_register_constructor – Auto-enregistrement • Signature C : **attribute**((constructor)) static void plugin_heuristics_register_constructor(void); • Utilité : Enregistre dynamiquement le greffon heuristique au chargement du programme.

==========
SECTION 8 : MOTEURS DE RESTITUTION UNIVERSELS (src/report/)
==========

Fonction 48 : report_cli.c – raptor_report_cli_print – Sortie console formattée • Signature C : void raptor_report_cli_print(const RaptorReportContext* ctx); • Utilité : Rendu textuel intégral du rapport dans le terminal avec codes ANSI pour les couleurs de sévérité et colonnage structuré.

Fonction 49 : report_html.c – raptor_report_html_generate – Export de rapport web • Signature C : bool raptor_report_html_generate(const RaptorReportContext* ctx, const char* filepath); • Utilité : Génère un rapport d'audit au format HTML souverain autonome avec thème sombre, visualisations interactives et tableaux filtrables.

==========
SECTION 9 : MOTEUR GRAPHIQUE ET CONTRÔLEURS DE VUES (src/ui/)
==========

Fonction 50 : ui_engine.c – raptor_ui_init – Initialisation graphique Raylib • Signature C : bool raptor_ui_init(int width, int height, const char* title); • Utilité : Configure la fenêtre Raylib, applique les dimensions, le titre, la synchronisation verticale (VSync) et la cible de 60 FPS.

Fonction 51 : ui_engine.c – raptor_ui_run – Boucle de surveillance active • Signature C : void raptor_ui_run(RaptorReportContext* ctx); • Utilité : Pilote la boucle de rendu principale, orchestre l'affichage des onglets et intègre le timer cyclique de 3s pour le rafraîchissement continu.

Fonction 52 : ui_engine.c – raptor_ui_shutdown – Libération graphique • Signature C : void raptor_ui_shutdown(void); • Utilité : Décharge les textures vectorielles (icônes) et détruit proprement la fenêtre graphique Raylib à la fermeture.

Fonction 53 : view_ai.c – view_ai_detect_os – Identification du système hôte • Signature C : static void view_ai_detect_os(char* out_name, size_t max_len); • Utilité : Interroge `/etc/os-release` pour afficher le nom exact de la distribution (ex: Linux Mint Debian Edition) dans le bandeau du rapport IA.

Fonction 54 : view_ai.c – view_ai_draw_pid_badge – Affichage badges d'alerte PID • Signature C : static float view_ai_draw_pid_badge(const char* role_title, const char* pid_val, float posX, float posY); • Utilité : Dessine un conteneur visuel haute visibilité (bordure jaune vif 2 px, fond sombre, texte blanc) mettant en évidence les identifiants cibles et parents.

Fonction 55 : view_ai.c – ui_view_ai_draw – Contrôleur du rapport IA [05] • Signature C : void ui_view_ai_draw(const RaptorReportContext* ctx); • Utilité : Consolide l'audit explicatif souverain, affiche le modèle Z-Score, trace la lignée du spawn et intègre le garde-fou strict interdisant le kill du PID 1.

Fonction 56 : view_processes.c – ui_view_processes_draw – Contrôleur Processus & RAM [02] • Signature C : void ui_view_processes_draw(RaptorReportContext* ctx); • Utilité : Affiche la table filtrable des processus, gère la sélection interactive, exécute la capture forensique, ordonne le signal de kill et consigne l'événement.

==========
SECTION 10 : POINT D'ENTRÉE DU SYSTÈME (src/main.c)
==========

Fonction 57 : main.c – main – Orchestrateur d'amorçage • Signature C : int main(int argc, char** argv); • Utilité : Parse les arguments de ligne de commande (`--cli` ou interface GUI), initialise le registre, lance le premier audit et passe le relais à l'UI.

==========
SECTION 11 : GREFFON RAPTOR NET-GUARD DÉFENSIF (src/plugins/plugin_net_guard.c)
==========

Fonction 58 : plugin_net_guard.c – net_guard_trim – Assainissement de chaîne • Signature C : char* net_guard_trim(char* str); • Utilité : Élimine les espaces blancs et sauts de ligne superflus en début et fin de chaîne pour fiabiliser le parsing des fichiers de règles.

Fonction 59 : plugin_net_guard.c – net_guard_validate_ip – Validation arithmétique IP & CIDR • Signature C : bool net_guard_validate_ip(const char* ip_str, uint32_t* out_ip, uint32_t* out_mask, uint16_t* out_port); • Utilité : Contrôle rigoureusement les plages IPv4 (0-255), extrait le port éventuel (1-65535) et calcule le masque binaire réseau à partir de la notation CIDR (/1 à /32).

Fonction 60 : plugin_net_guard.c – net_guard_load_file – Parsing tolérant de registre .conf • Signature C : bool net_guard_load_file(const char* filepath, bool is_custom_source); • Utilité : Lit ligne par ligne les fichiers de configuration, ignore les commentaires, segmente les champs par séparateur pipe (|) et peuple les tables mémoires DOM et IP.

Fonction 61 : plugin_net_guard.c – net_guard_init – Initialisation et fusion en RAM • Signature C : bool net_guard_init(void); • Utilité : Réinitialise les tables du registre et charge successivement le socle officiel (fw_officiel.conf) puis les ajouts locaux (fw_custom.conf) sans conflit.

Fonction 62 : plugin_net_guard.c – net_guard_add_custom_rule – Sérialisation et persistance IHM • Signature C : bool net_guard_add_custom_rule(NetGuardRuleType type, const char* target, const char* param, const char* desc); • Utilité : Formate avec espacements canoniques une nouvelle règle saisie dans l'IHM, l'écrit en mode ajout permanent dans fw_custom.conf et actualise la table active.

Fonction 63 : plugin_net_guard.c – net_guard_remove_rule – Suppression atomique de règle • Signature C : bool net_guard_remove_rule(NetGuardRuleType type, size_t index); • Utilité : Supprime une règle personnalisée selon son index mémoire et réécrit proprement l'intégralité du fichier fw_custom.conf sans toucher au registre officiel.

Fonction 64 : plugin_net_guard.c – net_guard_match_domain – Détection anti-combosquatting • Signature C : NetGuardVerdict net_guard_match_domain(const char* hostname, char* out_reason, size_t reason_sz); • Utilité : Analyse le nom de domaine demandé, vérifie la présence d'une marque protégée, valide les domaines officiels autorisés et lève l'alerte CRITIQUE si un mot-clé suspect y est accolé.

Fonction 65 : plugin_net_guard.c – net_guard_match_ip – Qualification binaire des endpoints IP • Signature C : NetGuardVerdict net_guard_match_ip(const char* remote_ip, uint16_t port, char* out_reason, size_t reason_sz); • Utilité : Applique le masque de sous-réseau sur l'adresse distante pour déterminer instantanément si le socket pointe vers une cible APPROUVÉ (vert) ou HOSTILE (rouge critique).

Fonction 66 : plugin_net_guard.c – net_guard_run_audit – Moteur d'audit périodique réseau • Signature C : bool net_guard_run_audit(RaptorReportContext* ctx); • Utilité : Orchestre la lecture de /proc/net/tcp et /proc/net/udp, confronte chaque connexion active aux tables de règles et peuple la section générique du rapport de sécurité.

Fonction 67 : plugin_net_guard.c – net_guard_cleanup – Déchargement et purge mémoire • Signature C : void net_guard_cleanup(void); • Utilité : Remet à zéro l'ensemble des structures de données du registre Net-Guard lors de la fermeture propre de l'application.

Fonction 68 : plugin_net_guard.c – net_guard_register – Auto-enregistrement constructeur GCC • Signature C : **attribute**((constructor)) void net_guard_register(void); • Utilité : Enregistre automatiquement la structure RaptorPlugin de Net-Guard auprès du bus central raptor_registry dès le chargement du binaire en mémoire.

Fonction 69 : plugin_net_guard.c – net_guard_update_official_rules – Rapatriement atomique GitHub • Signature C : bool net_guard_update_official_rules(void); • Utilité : Télécharge la dernière mouture de fw_officiel.conf via curl vers un fichier temporaire (.tmp), valide l'en-tête de conformité et applique un remplacement atomique POSIX (rename).

==========

SECTION 12 : RENSEIGNEMENT RÉSEAU, BGP & CDN INTEL (Fonctions 71 à 79) Module source : src/plugins/plugin_asn_intel.c En-tête : include/raptor/raptor_asn_intel.h Configuration : assets/rules/table_static.conf

==========

71. asn_intel_trim
    - Rôle : Nettoyage canonique des espaces blancs en tête et queue de chaîne.
    - Entrée : char* str
    - Sortie : char* (pointeur sur la chaîne assainie).
72. asn_intel_parse_cidr
    - Rôle : Décomposition arithmétique d'une notation CIDR (X.X.X.X/YY).
    - Calcul : Extraction de l'adresse réseau hôte et génération du masque binaire 32 bits.
    - Sortie : bool (validation syntaxique et mathématique).
73. asn_intel_load_file
    - Rôle : Parsing à chaud du fichier assets/rules/table_static.conf.
    - Fonctionnement : Peuplement dynamique de la table g_static_ranges sans recompilation.
    - Sortie : bool (succès du chargement).
74. asn_intel_match_static
    - Rôle : Évaluation arithmétique instantanée (0 ms) contre les infrastructures répertoriées.
    - Logique : (ip_host & r->netmask) == (r->network_ip & r->netmask).
    - Sortie : bool (vrai si correspondance CDN/Cloud trouvée).
75. asn_intel_query_bgp
    - Rôle : Sonde DNS TXT de secours vers le registre Team Cymru (origin.asn.cymru.com).
    - Extraction : Numéro ASN public et code pays ISO pour les adresses non répertoriées.
    - Sortie : bool (succès de la résolution distante).
76. asn_intel_lookup

- Rôle : Moteur central d'investigation en cascade.
- Cascade : 1. Cache dynamique -> 2. table_static.conf (0 ms) -> 3. Sonde DNS BGP.
- Sortie : bool (identification formelle de l'opérateur).

77. asn_intel_init
    - Rôle : Initialisation du greffon, allocation du verrou d'exclusion mutuelle et chargement des règles.
    - Sortie : bool (état opérationnel).
78. asn_intel_cleanup
    - Rôle : Vidange complète des structures et remise à zéro du cache en mémoire vive.
    - Sortie : void.
79. asn_intel_register
    - Rôle : Enregistrement constructeur GCC du greffon autonome sur le bus RaptorPlugin.
    - Sortie : void.

SECTION 13 : FORENSIC & TRAQUE DE MUTATIONS MÉMOIRE ANONYMES (R-X)

N°	Nom de la fonction	Fichier source	Signature	Rôle & Responsabilité technique
80	`mem_stealth_is_jit_whitelisted`	`src/core/mem_stealth.c`	`bool mem_stealth_is_jit_whitelisted(const char* comm)`	Isole les moteurs à compilation JIT légitimes (Chromium, Firefox, Electron, Node, JVM) pour éliminer les faux positifs structurels sur le code exécutable dynamique.
81	`mem_stealth_scan_maps_anon_rx`	`src/core/mem_stealth.c`	`bool mem_stealth_scan_maps_anon_rx(pid_t pid, const char* comm, StealthEvent* ev_out)`	Scanne chirurgicalement `/proc/[PID]/maps` à la recherche de plages mémoires possédant le droit d'exécution (`x`) avec un Inode à `0` (aucun fichier adossé sur disque), excluant `[vdso]` et `[vsyscall]` . Renseigne la structure d'incident `StealthEvent` .
82	`mem_stealth_dispatch_diff_event`	`src/core/mem_stealth.c`	`void mem_stealth_dispatch_diff_event(const StealthEvent* ev)`	Formate l'anomalie forensique identifiée et l'injecte dans le pipeline temporel du Cahier Différentiel (Onglet 4) pour marquage critique lors de la bascule d'état entre T-1 et T0.

Structure forensique associée (src/core/mem_stealth.h) :

typedef struct { pid_t pid; char comm[64]; unsigned long start_addr; unsigned long end_addr; size_t size_kb; char detail[128]; } StealthEvent;

SECTION 13 — TRAQUE MÉMOIRE ANONYME & MUTATIONS R-X (`src/core/mem_stealth.c`)

- **80.** `mem_stealth_whitelist_load`
    - **Prototype :** `bool mem_stealth_whitelist_load(const char* filepath)`
    - **Rôle :** Charge et analyse le fichier souverain externe (`assets/rules/mem_whitelist.conf`). Parse les directives `JIT` et `FFI` , nettoie les délimiteurs verticaux (`|`) et peuple le registre d'exception en mémoire vive (capacité portée à 256 motifs).
    - **Retour :** `true` si au moins une règle est chargée avec succès, sinon `false` .

- **81.** `mem_stealth_is_whitelisted`
  - **Prototype :** `bool mem_stealth_is_whitelisted(const char* comm, const char* exe)`
  - **Rôle :** Vérifie si un processus correspond à une signature autorisée dans la liste blanche externe. Effectue une comparaison insensible à la casse sur le nom court (`comm`) et le chemin absolu (`exe`) pour neutraliser les faux positifs (navigateurs, Flatpak, environnements de bureau Cinnamon/GNOME/KDE).
  - **Retour :** `true` si le binaire est légitime, sinon `false` .
- **82.** `mem_stealth_scan_maps_anon_rx`
  - **Prototype :** `bool mem_stealth_scan_maps_anon_rx(pid_t pid, const char* comm, const char* exe, StealthEvent* ev_out)`
  - **Rôle :** Analyse chirurgicale de `/proc/[PID]/maps` pour isoler les allocations anonymes rendues exécutables (`r-xp`) après bascule dynamique `mprotect()` . Isole les segments orphelins (Inode 0 sans fichier disque associé) hors vecteurs système protégés (`[vdso]` , `[vsyscall]`).
  - **Retour :** `true` si une mutation mémoire clandestine est constatée, sinon `false` .
- **83.** `mem_stealth_dispatch_diff_event`
  - **Prototype :** `void mem_stealth_dispatch_diff_event(const StealthEvent* ev)`
  - **Rôle :** Enregistre et déduplique les alertes forensiques dans le tampon circulaire de RAPTOR. Propulse les événements en temps réel vers le registre LIFO du Cahier Différentiel (Onglet 4).

SECTION 14 — DÉTECTION DES DÉTOURNEMENTS DU CHARGEUR DYNAMIQUE (`src/core/preload_hunter.c`)

- **84.** `preload_hunter_check_global`
  - **Prototype :** `bool preload_hunter_check_global(char* out_path, size_t out_len)`
  - **Rôle :** Audit d'intégrité passif de l'hôte via le contrôle d'accès direct à `/etc/ld.so.preload` . Identifie la persistance globale d'un rootkit en espace utilisateur forçant l'injection d'une bibliothèque partagée (`.so`) dans l'ensemble des exécutables de la machine.
  - **Retour :** `true` si le fichier existe et référence une bibliothèque active, sinon `false` .
- **85.** `preload_hunter_scan_process_env`
  - **Prototype :** `bool preload_hunter_scan_process_env(pid_t pid, char* out_preload_lib, size_t out_len)`
  - **Rôle :** Inspection unitaire et sécurisée du pseudo-fichier `/proc/[PID]/environ` . Parcourt les variables délimitées par des octets nuls (`\0`) pour intercepter la variable hostile `LD_PRELOAD=` ciblant un processus spécifique.
  - **Retour :** `true` en cas d'injection ciblée détectée avec extraction du chemin de la bibliothèque, sinon `false` .

SECTION 15 — NOTIFICATIONS BUREAU ASYNCHRONES FREEDESKTOP (`src/core/raptor_notify.c`)

- **86.** `raptor_notify_send_desktop`
  - **Prototype :** `void raptor_notify_send_desktop(const char* title, const char* body, bool is_critical)`
  - **Rôle :** Émet une bulle d'avertissement native vers le démon de notification du bureau via `notify-send` . Utilise un double découplage de processus (`fork()` / `_exit(0)`) garantissant 0 ms de blocage sur la boucle de rendu Raylib (60 FPS constants) et 0 processus zombie.
- **87.** `raptor_notify_evaluate_threats`
  - **Prototype :** `void raptor_notify_evaluate_threats(int current_threats, const char* primary_threat_name)`
  - **Rôle :** Moteur d'arbitrage d'alertes avec filtre anti-rebond temporel (délai de garde de 30 secondes). Déclenche instantanément la notification lors d'une transition critique (passage de 0 à N menaces) et notifie le retour au statut nominal lors de la résolution de l'incident.

SECTION 16 — MODE CLI DE SECOURS & OUTILS CHIRURGICAUX (`src/cli/`)

- **88.** `rap_dispatch_command` (`src/cli/rap_main.c`)
  - **Prototype :** `int rap_dispatch_command(int argc, char **argv, const RapConfig *cfg)`
  - **Rôle :** Micro-routeur central du binaire `rap` . Parcourt la table déclarative des greffons (`g_commands`) pour acheminer l'exécution vers la fonction dédiée (`status` , `top` , `hunt` , `rx` , `preload` , `net` , `kill` , `freeze` , `man`). Sans argument, bascule automatiquement vers l'affichage du manuel.
  - **Retour :** Code de sortie POSIX normalisé (`RAP_EXIT_OK` , `RAP_EXIT_THREAT` ou `RAP_EXIT_ERROR`).
- **89.** `rap_cmd_status` (`src/cli/cmd_status.c`)
  - **Prototype :** `int rap_cmd_status(int argc, char **argv, const RapConfig *cfg)`
  - **Rôle :** Diagnostic vital instantané de l'hôte en moins de 15 ms sans allocation dynamique. Lit `/proc/meminfo` (RAM totale et disponible), dénombre les processus actifs sous `/proc` et audite l'existence de l'altération globale `/etc/ld.so.preload` .
  - **Retour :** `RAP_EXIT_OK` (0) si le système est intègre, `RAP_EXIT_THREAT` (1) si `/etc/ld.so.preload` est présent.
- **90.** `rap_cmd_top` (`src/cli/cmd_top.c`)
  - **Prototype :** `int rap_cmd_top(int argc, char **argv, const RapConfig *cfg)`
  - **Rôle :** Analyse et ordonne les processus consommant le plus de mémoire vive résidente réelle (`VmRSS` dans `/proc/[PID]/status`). Élimine strictement les threads noyau (`rss == 0`) et applique un tri d'insertion statique borné dans la pile (0 octet sur le tas).

    - **Retour :** `RAP_EXIT_OK` (0) après affichage, `RAP_EXIT_ERROR` (2) si `/proc` est inaccessible.
- **91.** **`rap_cmd_hunt`** (`src/cli/cmd_hunt.c`)
    - **Prototype :** `int rap_cmd_hunt(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Moteur d'audit universel combiné. Coordonne séquentiellement la traque des mutations mémoires anonymes (`rap_cmd_rx`) et des détournements du chargeur dynamique (`rap_cmd_preload`) en une passe unique d'investigation.
    - **Retour :** `RAP_EXIT_THREAT` (1) si au moins une menace est identifiée par l'une des sondes, sinon `RAP_EXIT_OK` (0).
- **92.** **`rap_cmd_rx`** (`src/cli/cmd_rx.c`)
    - **Prototype :** `int rap_cmd_rx(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Traque chirurgicale dans `/proc/[PID]/maps` des segments mémoires virtuels anonymes convertis en exécutable (`r-xp`) avec un Inode à `0` sans ancrage fichier. Filtre automatiquement les faux positifs légitimes via `rap_is_mem_whitelisted` sauf si le drapeau `-v` (verbose) est activé.
    - **Retour :** `RAP_EXIT_THREAT` (1) en cas de mutation clandestine non autorisée, `RAP_EXIT_OK` (0) si le système est intègre.
- **93.** **`rap_cmd_preload`** (`src/cli/cmd_preload.c`)
    - **Prototype :** `int rap_cmd_preload(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Détection des rootkits userland via un audit à double palier : contrôle d'accès direct au fichier de persistance `/etc/ld.so.preload` et extraction sécurisée des variables d'environnement délimitées par des zéros binaires (`/proc/[PID]/environ`) pour intercepter `LD_PRELOAD=` .
    - **Retour :** `RAP_EXIT_THREAT` (1) en cas de détournement détecté, sinon `RAP_EXIT_OK` (0).
- **94.** **`rap_cmd_net`** (`src/cli/cmd_net.c`)
    - **Prototype :** `int rap_cmd_net(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Inspection passive directe de `/proc/net/tcp` sans dépendance externe (`netstat` / `ss`). Décode les adresses binaires IPv4 (little-endian), isole les sockets en communication active (`TCP_ESTABLISHED`) et qualifie chaque flux en temps réel via l'intelligence Net-Guard (`rap_net_eval_ip`).
    - **Retour :** `RAP_EXIT_THREAT` (1) si une connexion pointe vers une adresse IP hostile, sinon `RAP_EXIT_OK` (0).
- **95.** **`rap_cmd_kill`** (`src/cli/cmd_kill.c`)
    - **Prototype :** `int rap_cmd_kill(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Neutralisation cinétique chirurgicale d'un processus hostile par émission du signal non rattrapable `SIGKILL` . Valide préalablement la cible via `rap_check_safety_pid` et applique une demande de confirmation interactive sauf présence du drapeau `-f` (`--force`).
    - **Retour :** `RAP_EXIT_OK` (0) en cas de succès, `RAP_EXIT_ERROR` (2) en cas de refus de sécurité ou d'échec système.
- **96.** **`rap_cmd_freeze`** (`src/cli/cmd_kill.c`)
    - **Prototype :** `int rap_cmd_freeze(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Suspension conservatoire immédiate d'une menace via le signal `SIGSTOP` . Fige l'exécution de la cible en la plaçant dans l'état noyau `T` (*Stopped*) tout en préservant intacte sa mémoire virtuelle pour copie ou analyse forensique ultérieure.
    - **Retour :** `RAP_EXIT_OK` (0) si la cible est figée avec succès, `RAP_EXIT_ERROR` (2) en cas de refus de sécurité ou d'échec.
- **97.** **`rap_cmd_man`** (`src/cli/cmd_man.c`)
    - **Prototype :** `int rap_cmd_man(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Manuel de référence autonome embarqué intégralement en mémoire morte (`.rodata`). Restitue la documentation technique complète (syntaxe, commandes, codes de sortie POSIX, pipelines) et l'aide unitaire par sous-commande sans dépendre du disque, de `man` ou de `groff` .
    - **Retour :** Toujours `RAP_EXIT_OK` (0).
- **98.** **`rap_output_formatter`** (`src/cli/rap_common.c`)
    - **Prototype :** `void rap_output_formatter(const RapConfig *cfg, const char *c1, const char *c2, const char *c3, const char *c4)`
    - **Rôle :** Moteur transversal de formatage d'affichage sur 4 colonnes. Aligne proprement les colonnes avec espacement calibré en mode TTY interactif, ou génère un flux brut délimité par tabulations (`\t`) sans fioritures en mode redirection ou extraction machine (`-r` / `--raw`).
    - **Retour :** Aucun (`void`).
- **99.** **`rap_check_safety_pid`** (`src/cli/rap_common.c`)
    - **Prototype :** `int rap_check_safety_pid(const RapConfig *cfg, pid_t pid)`
    - **Rôle :** Barrière de sécurité inviolable prévenant les tirs fratricides et les arrêts critiques accidentels. Rejette catégoriquement le ciblage du PID 1 (`init` / `systemd`), du PID courant de l'utilitaire, du PPID (shell parent de l'opérateur) et des threads internes du noyau Linux (`maps` vide).
    - **Retour :** `0` si le PID est valide et autorisé, `-1` en cas d'interdiction de tir formelle.
- **100.** **`rap_parse_flags`** (`src/cli/rap_main.c`)
    - **Prototype :** `int rap_parse_flags(int *argc, char ***argv, RapConfig *cfg)`
    - **Rôle :** Analyseur d'arguments universels en ligne de commande. Extrait les drapeaux de combat (`-q` , `-r` , `-f` , `-v` , `-n` , `-h`), détecte si la sortie standard est connectée à un terminal (`isatty`) et réaligne le tableau `argv` pour isoler le verbe d'action à exécuter.
    - **Retour :** Toujours `0` .
- **107.** **`rap_is_mem_whitelisted`** (`src/cli/rap_common.c`)
    - **Prototype :** `int rap_is_mem_whitelisted(const char *comm)`

    - **Rôle :** Moteur d'assainissement passif des allocations dynamiques. Charge sans allocation dynamique (`0 heap`) les règles de `assets/rules/mem_whitelist.conf` (format `TYPE | MOTIF | DESC`), prend en compte la troncature `TASK_COMM_LEN` (15 octets) du noyau et écarte le bruit des moteurs JIT (Chromium) et des trampolines FFI (Cinnamon/GTK).
    - **Retour :** `1` si le processus figure dans la liste blanche légitime, sinon `0` .
- **108.** **`rap_net_eval_ip`** (`src/cli/rap_common.c`)
    - **Prototype :** `int rap_net_eval_ip(uint32_t ip, uint16_t port, char *out_desc, size_t desc_size)`
    - **Rôle :** Évaluateur CIDR autonome adossé aux registres souverains Net-Guard (`fw_officiel.conf` et `fw_custom.conf`). Intercepte instantanément les adresses IP répertoriées hostiles (relais C2/Tor), approuve les flux privés locaux (RFC 1918) et qualifie les communications sortantes standards (HTTPS:443, HTTP:80, DNS:53, SSH:22).
    - **Retour :** Constante entière de qualification (`RAP_NET_APPROVED` , `RAP_NET_STANDARD` , `RAP_NET_SUSPECT` , `RAP_NET_HOSTILE`).
- **109.** **`rap_cmd_heur`** (`src/cli/cmd_heur.c`)
    - **Prototype :** `int rap_cmd_heur(int argc, char **argv, const RapConfig *cfg)`
    - **Rôle :** Parcourt l'arborescence `/proc` pour traquer l'ensemble des anomalies comportementales et d'exécution clandestine. Résout le lien symbolique `/proc/[PID]/exe` de chaque tâche active, applique les filtres heuristiques experts (`rap_heur_check_process`) et projette les alertes formatées en console.
    - **Retour :** `RAP_EXIT_THREAT` (1) en cas d'anomalie détectée (binaire fantôme, zone volatile, usurpation), `RAP_EXIT_OK` (0) si le système est intègre, ou `RAP_EXIT_ERROR` (2) si `/proc` est inaccessible.
- **110.** **`rap_heur_check_process`** (`src/cli/cmd_heur.c`)
    - **Prototype :** `int rap_heur_check_process(pid_t pid, const char *comm, const char *exe_path, char *out_anomaly, size_t out_sz)`
    - **Rôle :** Évalue unitairement la posture de sécurité d'un processus via trois filtres défensifs sans allocation dynamique :
        1. Détection des binaires fantômes exécutés en mémoire vive après suppression sur le disque (`(deleted)`).
        2. Détection d'exécutions clandestines depuis des répertoires temporaires ou volatils suspects (`/tmp` , `/var/tmp` , `/dev/shm` , `/run/user/`).
        3. Détection d'usurpation d'identité système (*Masquerading*) sur les démons officiels (`systemd` , `sshd` , `cron` , `rsyslogd`) et faux threads `kworker` . Neutralise automatiquement les faux positifs lorsque `/proc/[PID]/exe` est inaccessible sans privilèges root.
    - **Retour :** `1` si une anomalie heuristique est avérée avec qualification dans `out_anomaly` , sinon `0` .